

Technische Beschreibung

Aufbau

Teil	Technik
Server	Python 3.12, FastAPI, Uvicorn; läuft als Hintergrundprozess auf einem Windows-PC
Datenbank	SQLite (eine Datei <code>daten/lager.db</code> , WAL-Modus), Zugriff über SQLAlchemy Core
Oberfläche	Server-generierte Seiten (Jinja2), HTMX, Alpine.js, Tailwind CSS; alle Dateien lokal, kein Internet nötig
Scanner-App	<code>/m</code> , installierbare Web-App (Manifest, Service Worker <code>/m/sw.js</code>); Scantaste über Tastatureingabe (DataWedge), Kamera-Scan über BarcodeDetector bzw. ZXing (lokal)
Offline-Modus	Warteschlange in IndexedDB des Geräts, Artikelkatalog <code>/m/katalog.json</code> zwischengespeichert, Übertragung über <code>/m/sync</code> mit UUID je Buchung (Tabelle <code>sync_log</code> verhindert Doppelbuchungen)
HTTPS	Port 8443; eigene kleine CA (<code>daten/lager_ca.crt</code> , Download <code>/zertifikat.crt</code>) stellt das Serverzertifikat aus und erneuert es bei geänderter IP/PC-Name automatisch
Etikettendruck	TSPL direkt an HPRT HT100 (Windows-Spooler RAW über winspool.drv oder TCP 9100); Fallback Browserdruck
Sicherung	täglich ZIP mit konsistenter SQLite-Kopie (Backup-API) und Anhängen

Gleichzeitige Buchungen (PC, mehrere Handscanner, API) werden über Schreibtransaktionen mit sofortiger Sperre (`BEGIN IMMEDIATE`) nacheinander ausgeführt; Lesen blockiert nicht. Getestet mit parallelen Entnahmen des letzten Teils.

Datenmodell

`artikel`, `lagerplaetze`, `bestand` (Menge je Artikel und Platz), `bewegungen` (jede Buchung mit Zeit, Benutzer, Menge $\pm$, Bestand danach, Empfänger, Kostenstelle, Zweck, Quelle `pc/mobil/api/import`), `reservierungen`, `bestellungen`, `inventuren/inventur_pos`, `lieferanten`, `anhaenge`, `users`, `audit`, `settings`.

Buchungsarten: `eingang`, `ausgang`, `umbuchung` (zwei verknüpfte Zeilen), `inventur`, `anlage`, `ausleihe`, `rueckgabe`, `info` (nur Historie aus dem Altsystem).

Alle Bestandsänderungen laufen über `app/services/lager.py` (Klasse `Lager`).

Übernahme aus Casper

`app/services/casper_import.py` liest den HeidiSQL-Export (nur INSERT-Anweisungen) und übernimmt:

Casper	Neu
<code>stammdaten.Artikelnummer / Bezeichnung</code>	<code>artikel.nummer / bezeichnung</code>
<code>Meldebestand / Mindestbestand</code> (Text, 0 = nicht gepflegt)	Zahl oder leer
<code>Freifeld1 „Lieferant“</code>	<code>lieferanten</code> (Schreibvarianten zusammengeführt)
<code>Freifeld2 „Gruppe“, Freifeld3 „Typenbezeichnung“</code>	<code>kategorie, typ</code>
<code>Freifeld4 „Austragungsgrund“, Freifeld5</code>	<code>notiz</code>
<code>lagerorte</code> (Leerzeichen bereinigt, Duplikate summiert, leer $\rightarrow$ OHNE-PLATZ)	<code>lagerplaetze + bestand</code>
<code>bewegungsdaten</code> (Eingang, Ausgang, Umbuchung/Umlagerung, Inventur, Neuer Artikel, Artikel bearbeitet/gelöscht)	<code>bewegungen</code> (<code>quelle = import</code>)
<code>benutzer</code>	<code>users</code> (gesperrt, ohne Passwort)

Lagerort-Zeilen zu nicht mehr vorhandenen Artikeln werden nicht übernommen (im Bericht genannt).

Schnittstelle

Token-Authentifizierung (`Authorization: Bearer <Schlüssel>`, Schlüssel je Benutzer unter *Einstellungen* $\rightarrow$ *Schnittstelle*). Endpunkte unter `/api/v1/...`: Artikel (Liste, Detail mit Beständen je Platz), Lagerplätze, Bewegungen seit ID, Buchung anlegen, Reservierung anlegen. Beschreibung maschinenlesbar unter `/api/openapi.json`. Gedacht für die spätere Anbindung der Instandhaltungs-Software (Ersatzteile reservieren/entnehmen je Auftrag).

Entwicklung

```
pip install -r requirements.txt
cd frontend && npm install && ./build_css.sh
python -m app.main # http://localhost:8080, https://localhost:8443
python -m pytest tests # LV_CASPER_EXPORT=<datei.sql> testet zusätzlich die Übernahme echter Daten
```